Practical: 8

Aim: Implementation of Graph and Searching (DFSand BFS).

Algorithm:

Steps
1. Start
2. Set NODES = 5 and Create graph[NODES] as an adjacency list
3. Define addEdge(u,v) and add v to graph[u] and u to graph[v]
4. Add the graph Edges
   addEdge(0,1), addEdge(0,2), addEdge(1,3), addEdge(1,4)
5. Define BFS(startNode) and initialize visited[NODES] = {false}
   and an Empty Queue q.
6. Set visited[startNode] = true and insert startNode into q
7. While q is not empty, get the front node using
   q.front(), remove it using q.pop() and display it
8. For each neighbour of the current node, if !visited[neighbour],
   marked it visited and inserted into q.
9. Define DFS((current, visited[]) and mark current as visited
   and display it
10. For each neighbour of Current, if visited[neighbour], recursively
    call DFS(neighbour, visited).
11. Call BFS(0) to perform BFS starting from node 0
12. Reset visited[NODES] = {false} for DFS
13. Call DFS(0, visited)
14. Display the BFS and DFS traversal results
15. Stop

program code:

```
#include<iostream>
#include <vector>
#include <queue>

using namespace std;

// Globalvariablesforsimplicity
constintNODES=5;              //Totalnumber of nodes (0 to 4)
vector<int>graph[NODES];

//Functiontoaddaconnectionbetween two nodes
void addEdge(int u, int v) {
   graph[u].push_back(v);
   graph[v].push_back(u);//Removethis line if you want a directed graph
}

//1.Breadth-FirstSearch(UsesaQueue)
void BFS(int startNode) {
   boolvisited[NODES]={false};//Track visited nodes
   queue<int>q;                //Queueto hold nodes to explore

   // Startwiththe initialnode
   visited[startNode] =true;
   q.push(startNode);

   cout << "BFS Traversal: ";
   while (!q.empty()) {
      int current = q.front();
      q.pop();
      cout << current << " ";

      //Lookatallneighborsofthecurrent node
      for(intneighbor:graph[current]) {
         if (!visited[neighbor]) {
            visited[neighbor] = true;
            q.push(neighbor);
         }
      }
   }
   cout << endl;
}
```

```
//2.Depth-FirstSearch(UsesRecursion / Implicit Stack)
void DFS(int current, bool visited[]) {
// Visit and print the current node
visited[current] = true;
cout << current << " ";


   //Instantlyjumpdeepintothefirstunvisited neighbor
   for (int neighbor : graph[current]) {
      if (!visited[neighbor]) {
         DFS(neighbor, visited);
      }
   }
}

int main() {
   // Create the graph connections
   addEdge(0, 1);
   addEdge(0, 2);
   addEdge(1, 3);
   addEdge(1, 4);


   // Run BFS
   BFS(0);

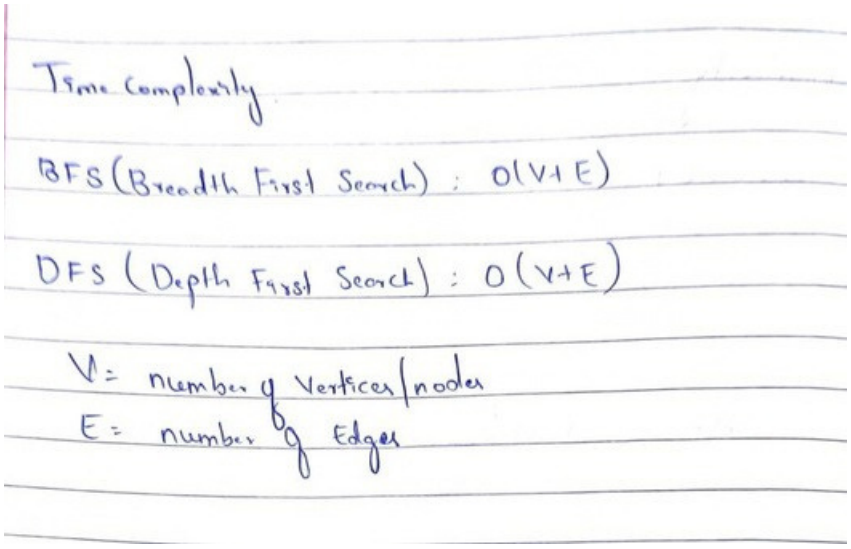
   // RunDFS
   boolvisited[NODES]={false};//Reset visited array for DFS
   cout << "DFS Traversal: ";
   DFS(0,visited);
   cout << endl;


   return 0;
}
```

Output:

```
PS C:\Users\THARUN\OneDrive\Documents\DAA(practicals)> cd "c:\Users\THARUN\OneDrive\Documents\DAA(practicals)\" ; if ($?) { g++ prac8.cpp -o prac8 } ; if ($?) { .\pra
c8 }
BFS Traversal: 0 1 2 3 4
DFS Traversal: 0 1 3 4 2
```

Time complexity:



Conclusion:
The programcorrectly traverses and displays all the reachable nodes using both